

Gerenciamento de Memória

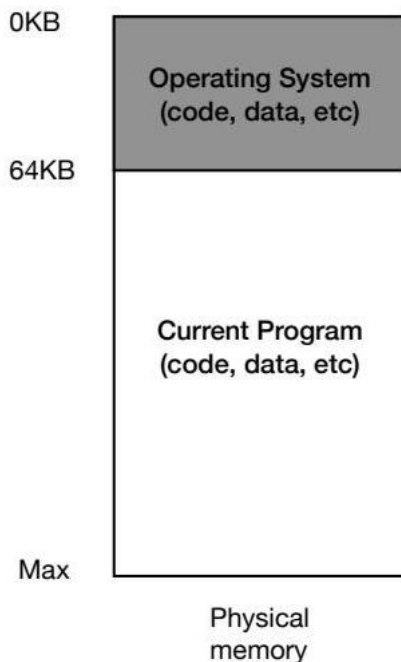
O Que Vamos Aprender?

- A memória física pode ser vista como um simples array de bytes.
- Programas precisam armazenar suas instruções e estruturas de dados na memória.
 - Como o SO abstrai e gerencia os acessos à memória feitos por programas?
 - Como é possível ter múltiplos programas compartilhando com segurança a mesma memória física?
 - Como o SO fornece a ilusão de ter mais memória do que a fisicamente disponível para executar programas?

Vamos estudar abstrações, mecanismos e estruturas!

Sistemas Antigos

O SO era uma biblioteca (conjunto de rotinas) colocada no início do espaço de memória (por exemplo, endereços de 0 a 64KB). O resto da memória era usado para executar o programa (um programa executando de cada vez...).



A imagem acima ilustra o layout da memória, onde a parte superior (64KB) é ocupada pelo sistema operacional e a parte inferior pelo programa atual.

A Era do Time Sharing

Com a chegada da multiprogramação e do time sharing:

- Os processos são agora trocados (por exemplo, enquanto esperam por E/S, para tornar o sistema mais interativo).
- As informações do processo devem ser carregadas nos registradores da CPU, mas fazê-lo a partir do disco é lento...

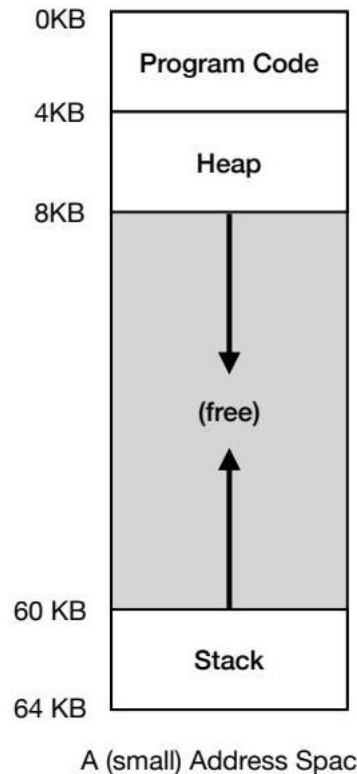
Solução: Informações importantes para executar/retomar processos que estão **prontos** para executar são mantidas na memória principal! *Memória física*.

Abstração de Memória

O Espaço de Endereçamento

O **Espaço de Endereçamento** contém o estado da memória de um processo.

- **Código**: as instruções do programa (o segmento de Código também é referido como o Texto na literatura).
- **Stack**: variáveis locais, argumentos de rotinas, valores de retorno...
- **Heap**: memória alocada dinamicamente (por exemplo, com `malloc()`).



A imagem exibe um diagrama do espaço de endereçamento, mostrando as seções de código do programa, heap, espaço livre e stack.

Heap e stack podem crescer enquanto o programa é executado. O espaço de endereçamento pode conter mais segmentos além do código, heap e stack. Por exemplo, o segmento de Dados contém dados estáticos, como variáveis globais inicializadas.

Tipos de Memória - API de Memória Stack

- Memória stack (também conhecida como memória automática).
 - Alocações e desalocações são tratadas pelo compilador.
 - No exemplo abaixo: quando a função `func()` é chamada, o compilador aloca espaço na stack para a variável `x`.
 - O compilador desaloca a memória quando a função retorna.

Tipos de Memória - API de Memória Heap

- Memória heap, memória dinâmica de longa duração.
 - Alocações e desalocações são tratadas pelo programador explicitamente.
 - No exemplo abaixo, ambos os tipos de alocação ocorrem.
 - O compilador aloca memória stack para um ponteiro para um inteiro (`int *x`).
 - Quando o programa chama `malloc()`, ele solicita espaço para um inteiro no heap.

Chamada de Malloc e Free - API de Memória

- `void *malloc(size_t size);`
 - Recebe, como argumento, o tamanho (quantidade) de memória a alocar (em bytes).
 - Retorna um ponteiro para o espaço recém-alocado (ou `NULL` se a chamada falhar).
 - O operador `sizeof()` pode ser usado para solicitar a quantidade correta de bytes para um determinado tipo de dado.
- `void free(void *ptr);`
 - Usado para liberar memória heap que não está mais sendo usada.
 - Recebe, como argumento, o ponteiro para uma região de memória alocada com `malloc()`.

Note que `malloc()` e `free()` podem ser chamados em funções distintas.

Erros Comuns de Malloc

- Esquecer de alocar memória pode levar a uma falha de segmentação.
 - Vamos aprender por que esse erro é chamado assim!
- Não alocar memória suficiente pode levar a um erro de estouro de buffer.
 - Erro complicado que pode se manifestar apenas em algumas execuções do programa.
- Esquecer de inicializar a memória alocada pode levar a bugs no programa.
 - Por exemplo, seu programa acaba lendo valores estranhos de um array de inteiros cujos valores não foram explicitamente inicializados (por exemplo, zerados).

Erros Comuns de Free

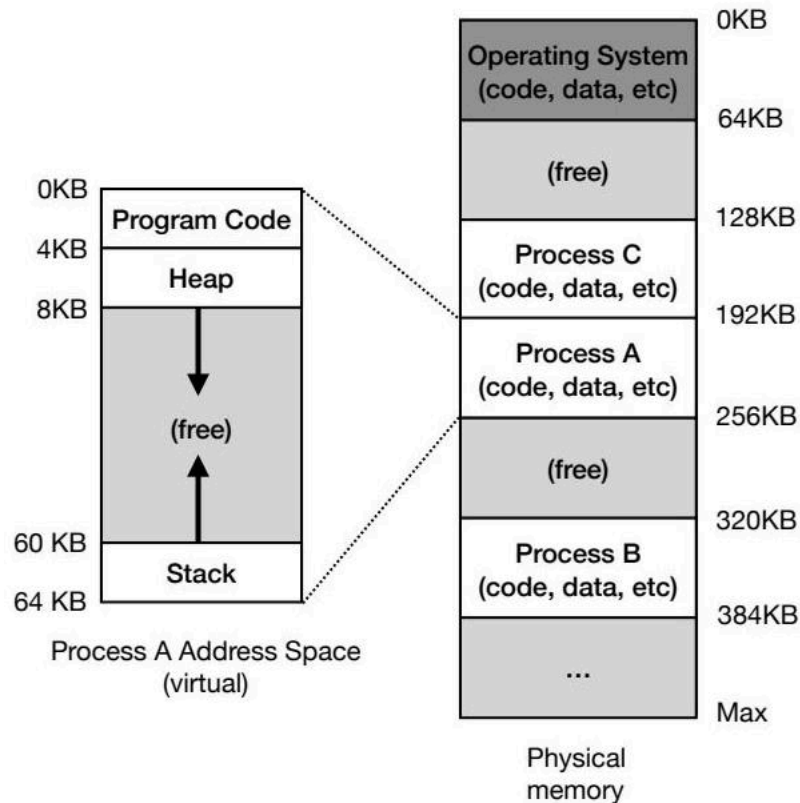
- Liberar memória antes de terminar de usá-la leva a um ponteiro pendente.
 - O ponteiro não está mais associado à memória alocada e, ao acessá-lo, o programa pode travar ou sobrescrever outra memória válida alocada entretanto.
- Liberar memória repetidamente leva a uma dupla liberação.
 - O resultado de uma dupla liberação é indefinido, o que significa que a biblioteca de alocação de memória pode exibir diferentes comportamentos errôneos.
- Chamar `free()` incorretamente pode levar a liberações inválidas.
 - Por exemplo, liberar outra memória alocada, travamentos do programa, ...

Chamadas de Sistema - API de Memória

- `malloc()` e `free()` não são chamadas de sistema, são chamadas de biblioteca.
 - Elas são construídas sobre chamadas de sistema como `brk` e `sbrk` que alteram o tamanho do heap (aumentando-o ou diminuindo-o).
 - Como programador, você não deve usar diretamente essas chamadas de sistema, use `malloc()` e `free()` em vez disso!
- `mmap()` é outra chamada para obter memória do SO.
 - Por enquanto, dê uma olhada na página [man](#), discutiremos esta chamada mais tarde...

Virtualização de Memória

Endereços Virtuais e Físicos



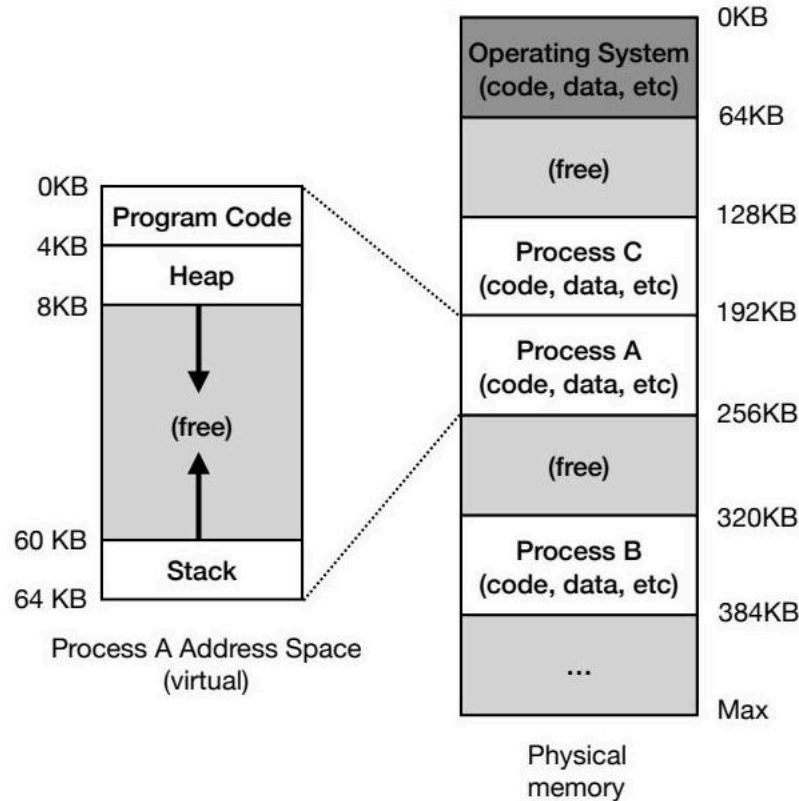
O diagrama ilustra como o espaço de endereçamento virtual do Processo A é mapeado para a memória física, mostrando código, heap e stack.

O espaço de endereçamento do Processo A não é realmente carregado na memória física nos endereços de 0 a 64KB. Quando o Processo A acessa o endereço 0 (endereço virtual), a solicitação é mapeada para o endereço 192KB na memória principal (endereço físico).

- Observação: Os espaços de endereçamento são muito maiores em sistemas reais.
 - 32 bits ($2^{32} = 4\text{GB}$) ou 64 bits ($2^{64} = 16\text{EB}$).
 - Ainda maior que a memória física do seu computador...

Como o SO constrói a abstração de um espaço de endereçamento privado, potencialmente grande, para múltiplos processos?

Objetivos do SO - Virtualização de Memória



O diagrama mostra o espaço de endereçamento virtual do Processo A e sua alocação correspondente na memória física.

- **Transparência:** Os processos não estão cientes de que sua memória está sendo virtualizada. Cada processo se comporta como se tivesse sua própria memória física privada.
 - Os endereços que você vê como programador (ou programa de nível de usuário) são virtuais!
- **Eficiência:** A virtualização de memória deve ser eficiente em termos de desempenho (ou seja, os programas não podem ser executados mais lentamente) e espaço (ou seja, as estruturas usadas pelo SO não podem consumir muita memória).
- **Proteção:** Os processos não podem acessar/modificar regiões de memória de outros processos ou do SO (ou seja, os espaços de endereçamento devem ser isolados).
 - O isolamento é uma propriedade chave para construir sistemas confiáveis.

Tradução de Endereços

- **Objetivo:** Criar a ilusão de que cada processo tem sua própria memória privada.
- **Solução:** Cada acesso à memória (por exemplo, fetch, load, store) feito por um processo usa um endereço virtual que deve ser traduzido, pelo hardware, para o endereço físico correspondente (ou seja, localização real na memória).
- O hardware fornece os mecanismos de baixo nível para traduzir eficientemente os endereços...
- O SO é responsável por instruir o hardware a fazer as traduções corretas, gerenciando a memória e protegendo os processos uns dos outros.

Tradução de Endereços - Algumas Suposições Iniciais

Vamos definir algumas suposições para raciocinar sobre a virtualização de memória:

1. O espaço de endereçamento de um processo é colocado contiguamente na memória física.
2. O tamanho de um espaço de endereçamento é menor que o tamanho da memória física.
3. Cada espaço de endereçamento tem exatamente o mesmo tamanho.

A maioria dessas suposições não é realista... ...vamos refiná-las à medida que avançamos.

Registrador Base e Limites (aka Relocação Dinâmica)

- O processador (CPU) tem dois registradores adicionais, a base e os limites.
 - A Unidade de Gerenciamento de Memória (MMU) é o componente da CPU responsável por manter esses registradores e ajudar na tradução de endereços.
- A ilusão de virtualização de memória criada pelo SO consiste em:
 - Cada programa é escrito e compilado como se fosse carregado no endereço 0.
 - Quando o programa começa a ser executado, o SO decide onde carregá-lo na memória física, ou seja, define o registrador base para o endereço de memória física.
 - Para cada acesso à memória feito pelo programa, os endereços virtuais são traduzidos em físicos, pelo processador, da seguinte forma:

$$\text{Physicaladdress} = \text{Virtualaddress} + \text{Base}$$

Registrador de Limites - Base e Limites (aka Relocação Dinâmica)

O registrador de limites ajuda a proteger os limites do espaço de endereçamento de cada processo. O processador primeiro verifica se o acesso à memória está dentro dos limites. Se não estiver, uma exceção de fora dos limites (trap!) é gerada. O SO deve fornecer tratadores de exceção para este trap.

Como a tradução de endereços é feita em tempo de execução, o espaço de endereçamento de um processo pode ser dinamicamente realocado (movido) enquanto o processo está sendo executado:

1. O SO deve desescalonar o processo.
2. Copiar seu espaço de endereçamento para outro local na memória física.
3. Atualizar os registradores base e limites.
4. Executar o processo.

Os registradores base e limites devem ser salvos e alterados quando os processos são trocados.

Exemplo - Base e Limites (aka Relocação Dinâmica)

Exemplo: O Processo A tem um espaço de endereçamento de 4096 bytes (4KB) carregado no endereço físico 16348 (16 KB). Se o valor do endereço virtual for maior que o registrador de limites ou negativo (4096), o acesso está fora dos limites. Caso contrário, adicione a base (16384) ao endereço virtual para obter o físico.

Virtual Address		Physical Address
0	→	16384 (16 KB)
1024	→	17408 (17 KB)
3000	→	19384
4400	→	FAULT (20784 IS OUT OF BOUNDS)

A imagem mostra um exemplo de tradução de endereços, com endereços virtuais mapeados para seus correspondentes endereços físicos.

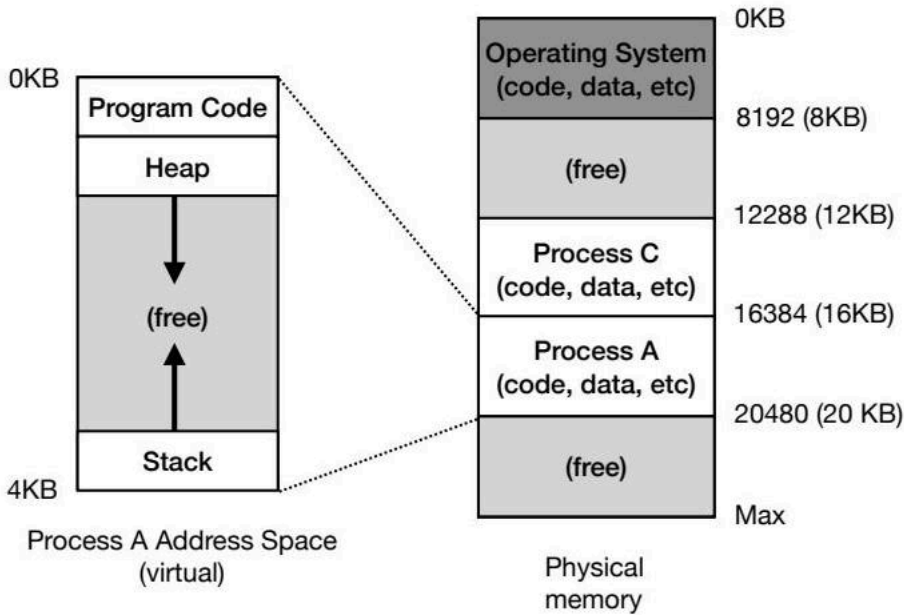


Diagrama ilustrando a relação entre memória virtual e física, com o espaço de endereçamento do Processo A e sua alocação na memória física.

Execução Direta Limitada com Relocação Dinâmica

O SO (Modo Kernel) cria o PCBA, encontra um slot de memória livre na lista livre, define os registradores base/limite e retorna do trap. O Hardware restaura os registradores do Processo A, move para o modo de usuário e pula para o PC do Processo A.

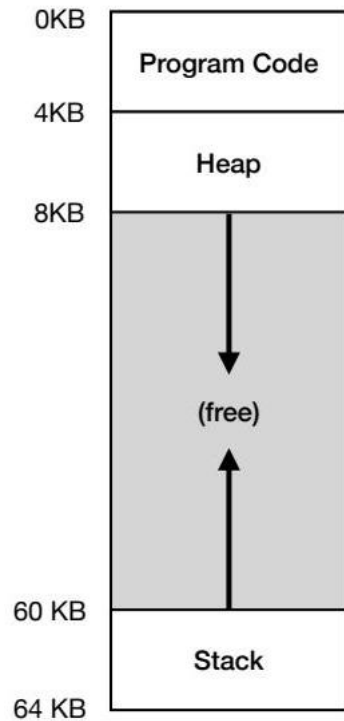
O Processo (Modo Usuário) executa `main()`, etc. O Hardware garante que o endereço esteja dentro dos limites, traduz o endereço virtual para o endereço físico, garante que o endereço esteja dentro dos limites (fora dos limites!), move para o modo kernel e pula para o tratador de trap.

O SO lida com o trap, decide terminar o Processo A, desaloca a memória do Processo A, adiciona-a à lista livre e remove o PCBA.

Se estes conceitos parecerem confusos, reveja os slides sobre Mecanismos de Virtualização da CPU!

Fragmentação Interna - Heap e Stack

Existe um bloco de memória não utilizado potencialmente grande entre o heap e a stack que ainda está ocupando memória física. Por exemplo, um espaço de endereçamento de 32 bits requer 4 GB de memória principal (enquanto muitos programas podem realmente usar apenas alguns MBs...).



A (small) Address Space

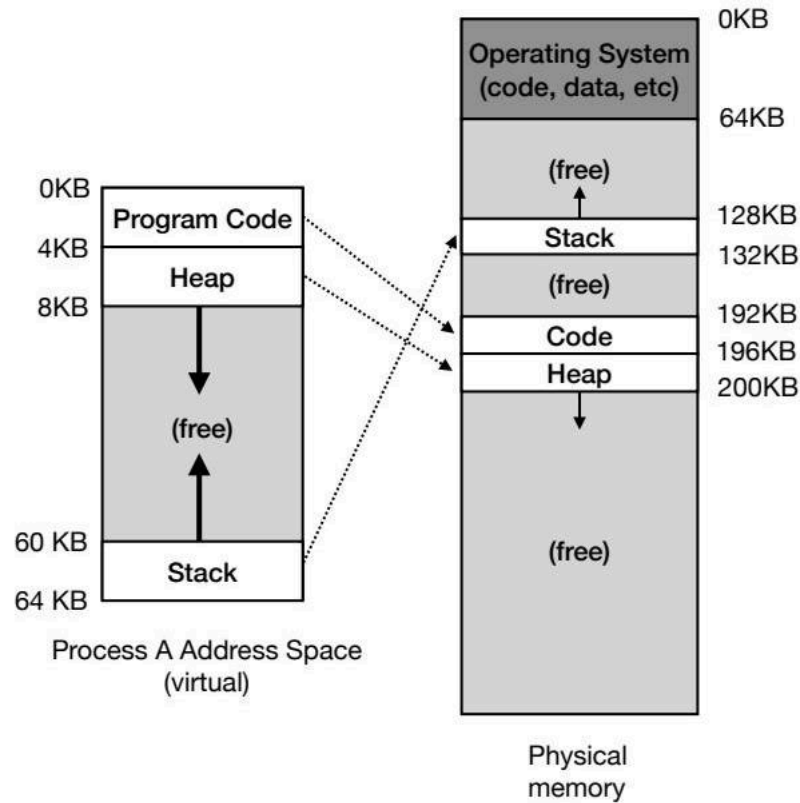
O diagrama mostra a fragmentação interna no espaço de endereçamento, com uma grande área livre entre o heap e a stack.

Em outras palavras, colocar o espaço de endereçamento contiguamente na memória física leva à **fragmentação interna**, ou seja, memória desperdiçada dentro do espaço de endereçamento de um processo.

Como o SO pode reduzir essa fragmentação interna?

Segmentação - Base/Limites Generalizados

Vamos relaxar a Suposição 1 - O espaço de endereçamento do processo é colocado contiguamente na memória física.



A imagem ilustra a segmentação, mostrando como diferentes segmentos do espaço de endereçamento virtual são mapeados para diferentes regiões da memória física.

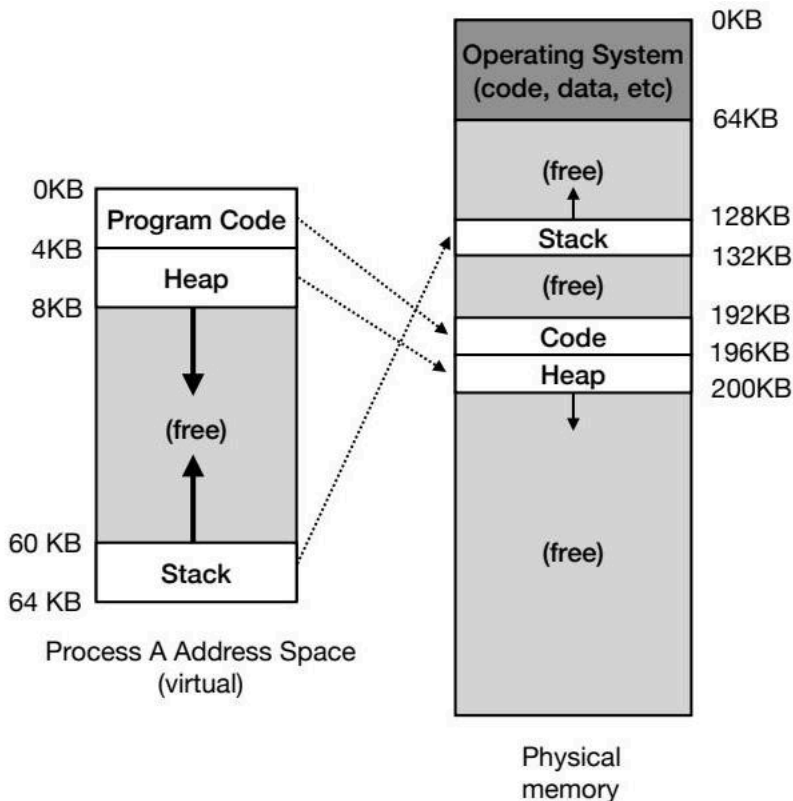
Cada segmento lógico do espaço de endereçamento (por exemplo, código, heap, stack) é colocado em uma região de memória física diferente. Esta técnica, chamada **segmentação**, pode ser "facilmente" implementada se houver vários registradores base e limites, um por segmento lógico. Cada segmento pode ser realocado na memória à medida que o sistema é executado. A troca de contexto agora requer salvar e restaurar todos esses registradores.

Exemplos de Segmentação

Exemplo 1: Solicitação para o endereço virtual 100 (segmento de código).

Physical address = 196 608 (base) + 100 (offset) = 196 708

Segmento	Registrador Base	Registrador de Limites
CODE	196 608 (192KB)	4096 (4KB)
HEAP	200 704 (196KB)	4096 (4KB)
STACK	135 168 (132KB)	4096 (4KB)



O diagrama mostra como os segmentos de código, heap e stack no espaço de endereçamento virtual são mapeados para suas respectivas localizações na memória física.

Exemplos - Segmentação

Exemplo 1: Solicitação para o endereço virtual 100 (segmento de código).

Physical address = 196 608 (base) + 100 (offset) = 196 708

Segmento	Registrador Base	Registrador de Limites
CODE	196 608 (192KB)	4096 (4KB)
HEAP	200 704 (196KB)	4096 (4KB)
STACK	135 168 (132KB)	4096 (4KB)

Exemplo 2: Solicitação para o endereço virtual 5120 (segmento heap). Se adicionarmos o endereço virtual ao registrador base do heap, obtemos:

Physical address = 200 704 + 5120 = 205 824 (201KB - endereço errado!)

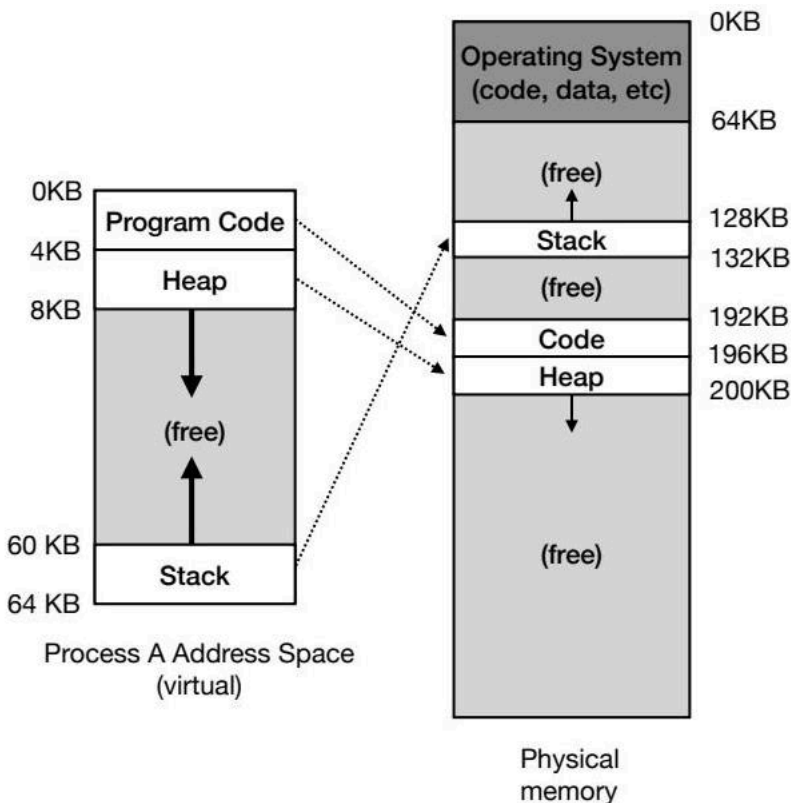


Diagrama ilustrando como o espaço de endereçamento virtual é mapeado para a memória física com a segmentação.

Exemplos - Segmentação

Exemplo 1: Solicitação para o endereço virtual 100 (segmento de código).

Physical address = 196 608 (base) + 100 (offset) = 196 708

Segmento	Registrador Base	Registrador de Limites
CODE	196 608 (192KB)	4096 (4KB)
HEAP	200 704 (196KB)	4096 (4KB)
STACK	135 168 (132KB)	4096 (4KB)

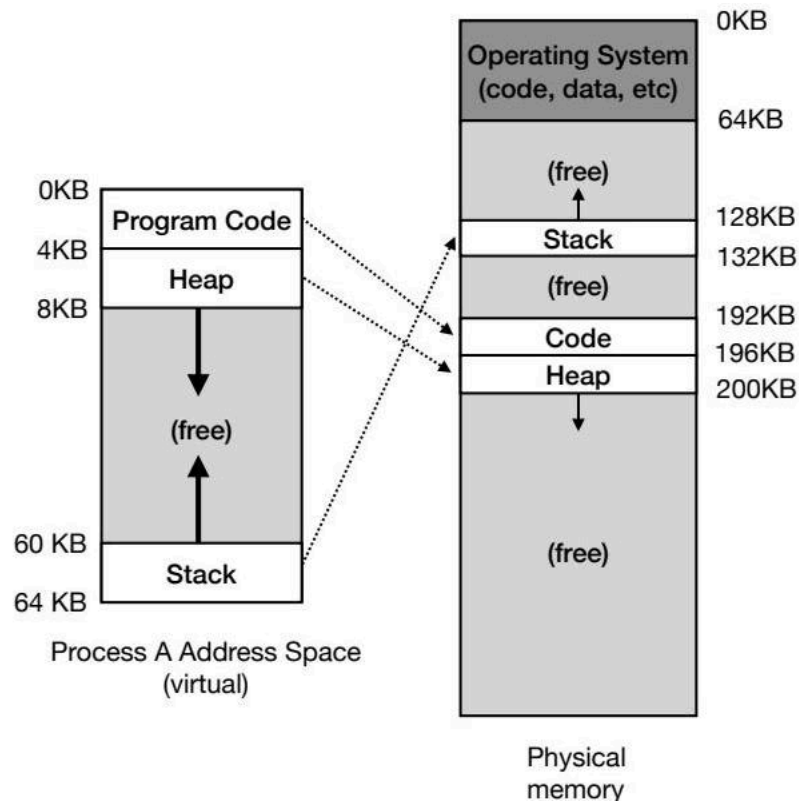
Exemplo 2: Solicitação para o endereço virtual 5120 (segmento heap). Se apenas adicionarmos o endereço virtual ao registrador base do heap, obtemos:

- Primeiro extraímos o offset para o heap.
- Então calculamos o endereço físico.

Offset = 5120 (endereço virtual) - 4096 (início do endereço virtual do heap) = 1024

Physical address = 200 704 (base) + 1024 (offset) = 201 728 (197KB)

Physical address = 200 704 + 5120 = 205 824 (201KB - endereço errado!)



O diagrama ilustra o mapeamento entre os segmentos do espaço de endereçamento virtual (código, heap, stack) e suas localizações correspondentes na memória física.

Exemplos - Segmentação

Exemplo 1: Solicitação para o endereço virtual 100 (segmento de código).

Physical address = 196 608 (base) + 100 (offset) = 196 708

Segmento	Registrador Base	Registrador de Limites
CODE	196 608 (192KB)	4096 (4KB)
HEAP	200 704 (196KB)	4096 (4KB)
STACK	135 168 (132KB)	4096 (4KB)

Exemplo 2: Solicitação para o endereço virtual 5120 (segmento heap). Se apenas adicionarmos o endereço virtual ao registrador base do heap, obtemos:

- Primeiro extraímos o offset para o heap.
- Então calculamos o endereço físico.

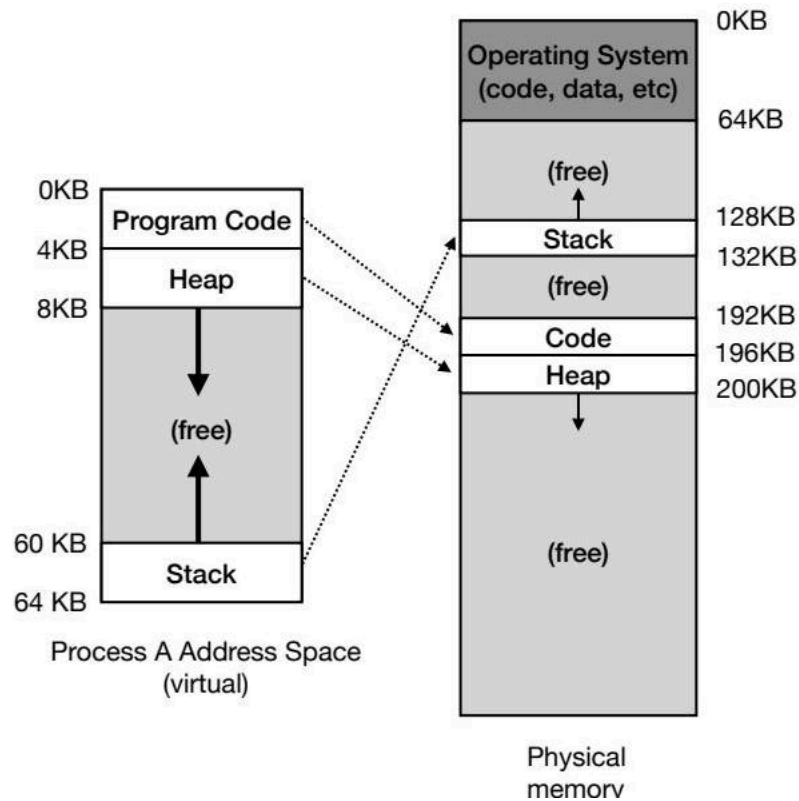
Physical address = 200 704 + 5120 = 205 824 (201KB - endereço errado!)

Physical address = 200 704 (base) + 1024 (offset) = 201 728 (197KB)

Offset = 5120 (endereço virtual) - 4096 (início do endereço virtual do heap) = 1024

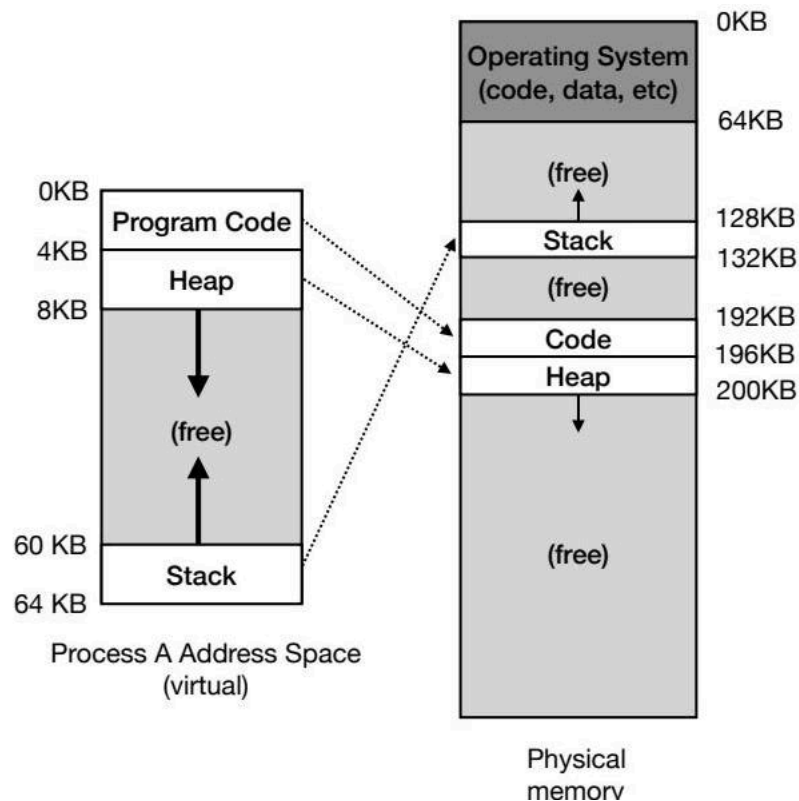
Exemplo 3: Solicitação para o endereço virtual 9216 (9KB) (segmento heap).

Offset = 9216 - 4096 = 5120 (maior que 4096 - fora dos limites!)



A imagem mostra um diagrama de alocação de memória com segmentação, destacando as áreas de código, heap, stack e espaço livre na memória virtual e física.

O hardware detecta que o endereço está fora dos limites, traps para o SO, e este último termina o processo infrator. Agora você sabe a origem de uma [falha de segmentação](#)!



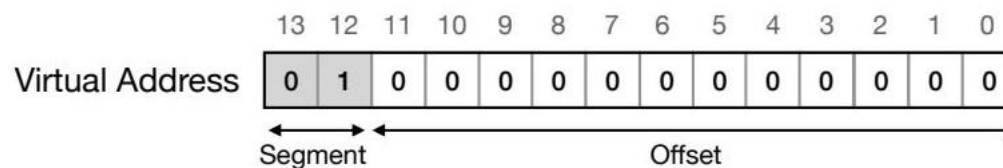
A imagem ilustra a segmentação, mostrando como diferentes segmentos do espaço de endereçamento virtual são mapeados para diferentes regiões da memória física.

Segmentação de Tradução de Hardware

Segmento	Registrador Base	Registrador de Limites
CODE (00)	196 608 (192KB)	4096 (4KB)
HEAP (01)	200 704 (196KB)	4096 (4KB)
STACK (10)	135 168 (132KB)	4096 (4KB)

Como o hardware sabe o segmento e o offset de um endereço virtual? Solução: use os bits superiores do endereço virtual para identificar o segmento.

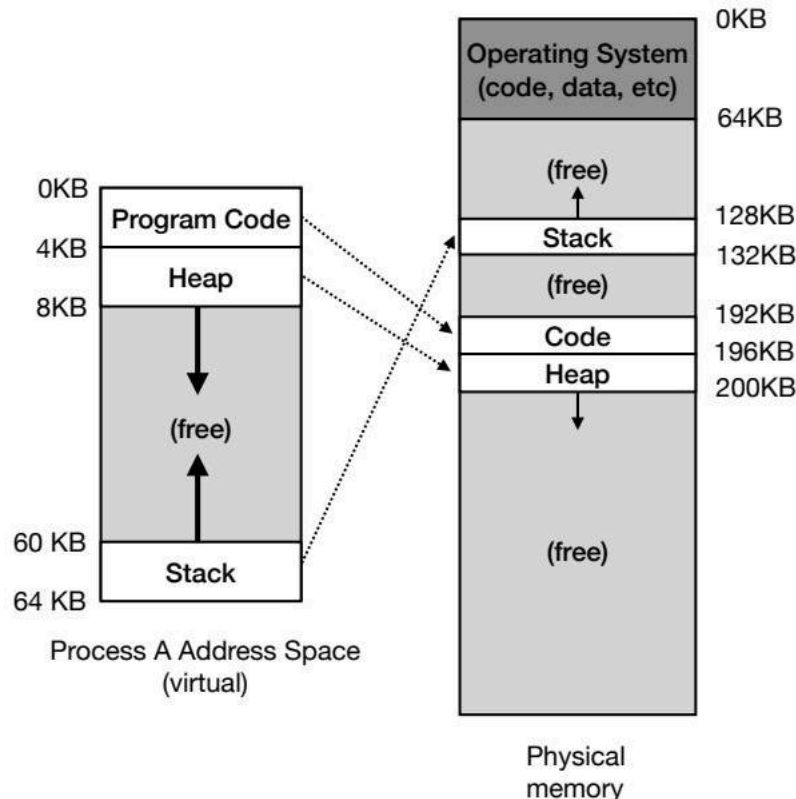
Exemplo: considerando um endereço virtual com 14 bits.



A imagem mostra a estrutura de um endereço virtual de 14 bits, dividido em segmento e offset.

Acesso ao segmento heap no offset 0:

Physical address = 200 704 (base) + 0 (offset) = 200 704



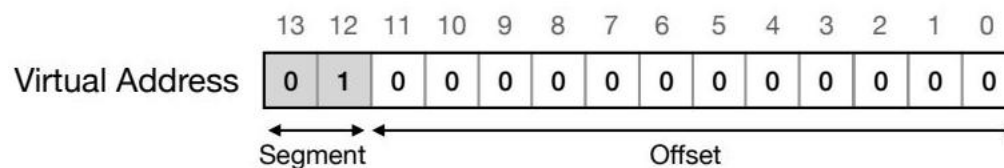
A imagem ilustra a segmentação, mostrando como os segmentos do espaço de endereçamento virtual (código, heap, stack) são mapeados para diferentes localizações na memória física.

Segmentação de Tradução de Hardware

Segmento	Registrador Base	Registrador de Limites
CODE (00)	196 608 (192KB)	4096 (4KB)
HEAP (01)	200 704 (196KB)	4096 (4KB)
STACK (10)	135 168 (132KB)	4096 (4KB)

Como o hardware sabe o segmento e o offset de um endereço virtual? Solução: use os bits superiores do endereço virtual para identificar o segmento.

Exemplo: considerando um endereço virtual com 14 bits.



A imagem mostra a estrutura de um endereço virtual de 14 bits dividido em um segmento e um offset.

Acesso ao segmento heap no offset 0:

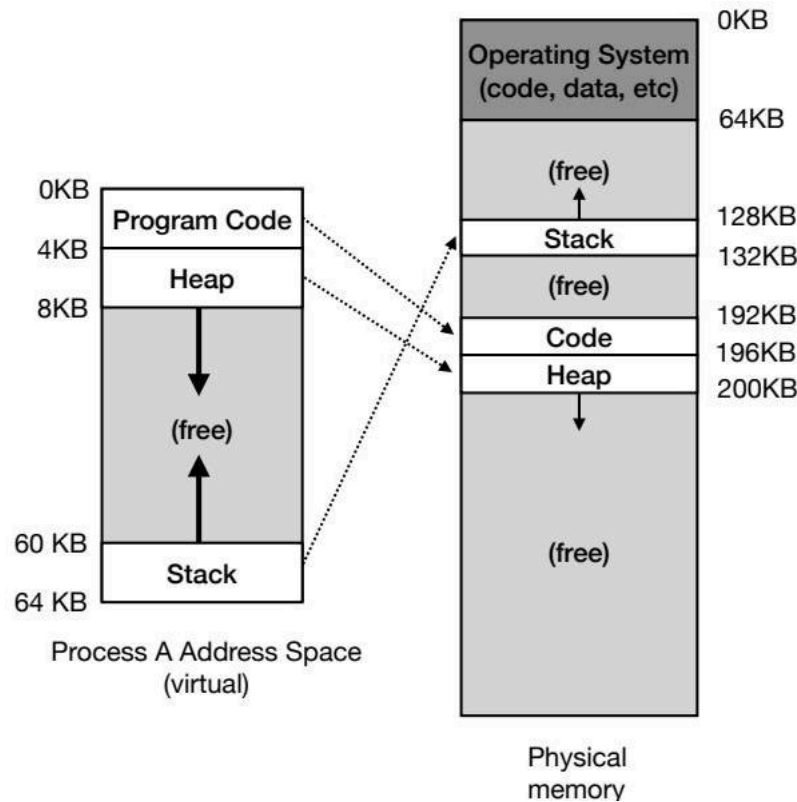
Physical address = 200 704 (base) + 0 (offset) = 200 704

Outro exemplo: Acesso ao segmento de código no offset 100:

 Diagrama de um endereço virtual de 14 bits

A imagem mostra a estrutura de um endereço virtual de 14 bits, com um segmento e um offset.

Physical address = 196 608 (base) + 100 (offset) = 196 708



A imagem ilustra a segmentação, mostrando como os segmentos do espaço de endereçamento virtual (código, heap, stack) são mapeados para diferentes localizações na memória física.

Segmentação - Tradução Stack

Note que a stack começa no endereço 132 KB.

Segmento	Registrador Base	Registrador de Limites	Cresce Positivo?
CODE (00)	192KB	4KB	1
HEAP (01)	196KB	4KB	1
STACK (10)	132KB	4KB	0

A stack cresce para trás! Ou seja, vai do endereço físico 132KB para 128KB no exemplo.

Exemplo: Acesso ao endereço virtual 63KB:

 Diagrama da relação entre memória virtual e física

A imagem mostra um diagrama de alocação de memória com segmentação, destacando as áreas de código, heap, stack e espaço livre na memória virtual e física.

Cálculo do Deslocamento (Offset)

- Para calcular o **offset**, subtraímos o endereço base do endereço virtual. $Offset = Endere\c{c}oVirtual - Endere\c{c}oBase$
- Exemplo: $Offset = 63KB(endere\c{c}ovirtual) - 64KB(iniciodoendere\c{c}ovirtualdapilha) = -1KB$
- O endereço físico é então calculado usando o offset. $Endere\c{c}oFisico = Base - Offset$
- Exemplo: $Endere\c{c}oFisico = 132KB(base) - 1KB(offset) = 131KB$
- O sistema operacional (SO) necessita do auxílio do hardware para traduzir os endereços da pilha.
- Um bit extra é definido como 1 se o segmento cresce em direção positiva e 0 caso contrário.

Suporte ao Compartilhamento na Segmentação

A **segmentação** possibilita o compartilhamento de segmentos entre múltiplos processos.

- Exemplo: Segmentos de código de bibliotecas usadas por vários programas podem ser compartilhados.
- Se os registradores base para os segmentos de código dos Processos A e B apontarem para o mesmo endereço físico, temos um segmento compartilhado.
- Importante: Cada processo tem seus próprios registradores. O compartilhamento é alcançado fazendo com que esses registradores se refiram ao mesmo endereço físico.
- São necessários bits de proteção (suporte adicional do hardware).
 - Esses bits indicam o que um processo pode fazer com cada segmento (ler, escrever ou executar instruções).
 - Ao marcar um segmento como somente leitura, vários processos podem usá-lo sem comprometer o isolamento.
- O hardware verifica se o acesso ao segmento tem as permissões corretas. Caso contrário, o hardware gera uma exceção e o SO lida com o processo infrator.

Segmento	Registro Base	Registro de Limites	Cresce Positivo?	Proteção
CODE	192KB	4KB	1	READ-EXECUTE
HEAP	196KB	4KB	1	READ-WRITE
STACK	132KB	4KB	0	READ-WRITE

Considerações Finais sobre Segmentação

Na **segmentação**, relaxamos algumas premissas iniciais:

1. O espaço de endereço do processo é colocado de forma contígua na memória física.
2. O tamanho de um espaço de endereço é menor do que o tamanho da memória física.
3. Cada espaço de endereço tem exatamente o mesmo tamanho.
 - No exemplo, há apenas três segmentos (segmentação de grão grosso).
 - É possível ter segmentos menores (segmentação de grão fino).
 - Isso pode exigir suporte adicional, como uma tabela de segmentos (armazenada na memória), para traduzir os segmentos para seus locais na memória física.

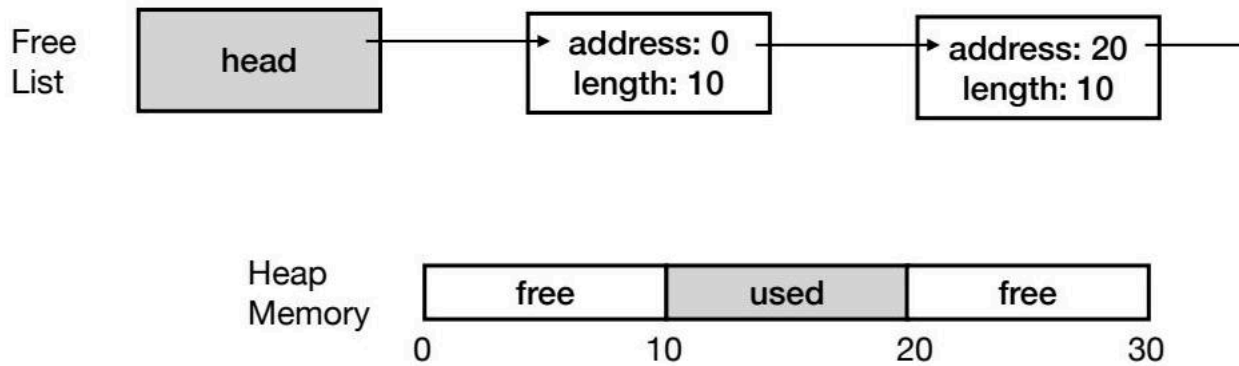
Gerenciamento do Espaço Livre

Quando um novo espaço de endereço é criado, o SO deve saber onde colocar os segmentos correspondentes (ou seja, na memória física).

- Como os segmentos variam em tamanho, a memória física pode rapidamente ser preenchida com vários pequenos buracos, levando à **fragmentação externa**.

Fragmentação Externa: Ocorre quando há espaço total de memória livre suficiente para atender a uma solicitação de alocação, mas esse espaço não é contíguo.

- Exemplo: Se um processo precisa alocar 20 bytes de memória, não há um pedaço contíguo de memória livre para fazer isso. A memória está fragmentada externamente em dois pedaços de 10 bytes.



A imagem acima ilustra a fragmentação externa, onde a memória está dividida em blocos livres e usados, dificultando a alocação de um bloco contíguo grande o suficiente para a solicitação de um processo.

- Interromper processos para realocar e compactar sua memória é uma estratégia dispendiosa.
- A abordagem típica é, em vez disso, usar uma estratégia de gerenciamento de lista livre.

Desafios no Gerenciamento do Espaço Livre

- É fácil quando a memória que está sendo gerenciada é alocada com um tamanho fixo. (Aguarde nossa discussão sobre paginação).
- É mais difícil e interessante quando se está usando unidades de tamanho variável.
 - Útil para segmentação (como discutido no slide anterior).
 - Também útil para alocação de memória heap em nível de usuário com `malloc()` e `free()`.

Nos exemplos a seguir, focaremos neste segundo caso de uso.

- Desafios:
 - Redução da fragmentação externa.
 - Eficiência (ou seja, encontrar rapidamente um pedaço livre para alocar memória).

A Estrutura da Lista Livre

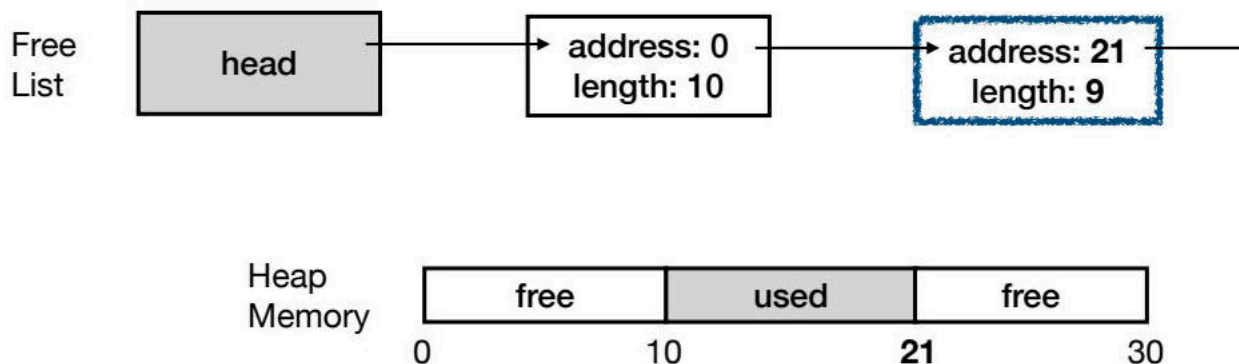
Uma **lista livre** contém os endereços e o comprimento dos pedaços livres na memória física.

- No exemplo abaixo:
 - Uma solicitação `malloc()` maior que 10 bytes não pode ser atendida, retornando `NULL` (ou seja, não há um pedaço contíguo de memória maior que 10 bytes).
 - Uma solicitação de 10 bytes é atendida por um dos pedaços livres.

Mecanismo de Divisão (Splitting) na Lista Livre

E se houver uma solicitação para alocar 1 byte?

- Um dos pedaços é dividido em dois pedaços. É por isso que a técnica é conhecida como **divisão** (splitting).



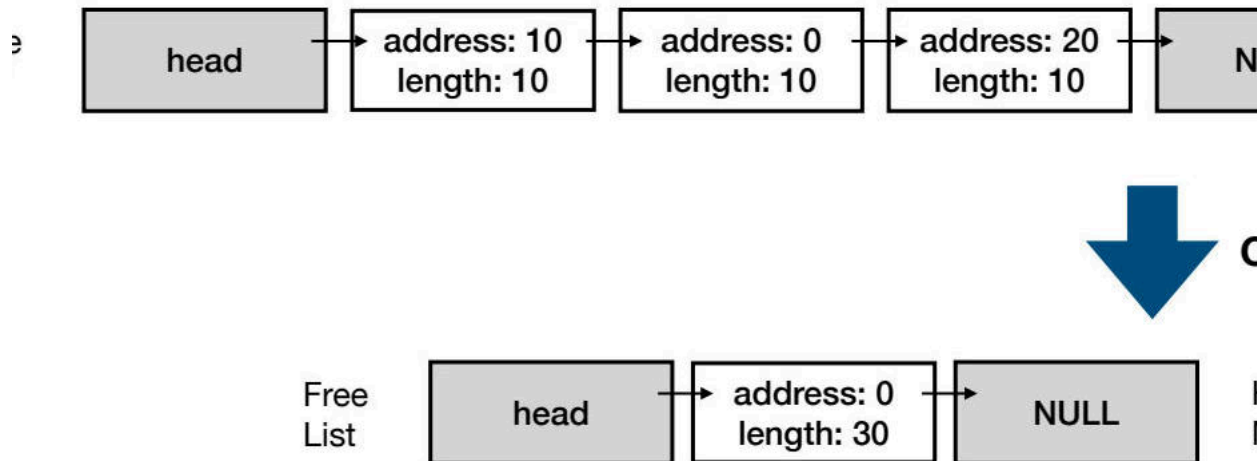
A imagem acima mostra como um bloco de memória é dividido para atender a uma solicitação de alocação, resultando em um bloco usado e um novo bloco livre menor.

- Supondo que o segundo pedaço livre seja escolhido:
 - `malloc()` retorna o endereço 20, e o segundo pedaço livre agora começa no endereço 21.

Mecanismo de Coalescência (Coalescing) na Lista Livre

E se o programa eventualmente liberar toda a memória alocada e...

- A lista livre acabar dividida em três pedaços de 10 bytes, não sendo capaz de atender a solicitações `malloc()` maiores que 10 bytes?
- Outro mecanismo, chamado **coalescência**, é usado para mesclar pedaços contíguos livres.



A imagem acima ilustra como blocos de memória adjacentes são combinados em um único bloco maior, aumentando a disponibilidade de memória contígua.

Incorporação da Lista Livre no Espaço Heap

Onde a lista livre é armazenada?

- Opção 1: Em outra região de memória física.
- Opção 2: Incorporada no espaço heap.

Supondo um heap vazio com 4KB, começando no endereço virtual 16384 (16KB):

- Começamos com um pedaço, incluindo um cabeçalho com 8 bytes:
 - 4 bytes para o tamanho do espaço livre no pedaço.
 - 4 bytes apontando para o próximo pedaço. Se o valor for `NULL` (representado pelo valor 0), então estamos no final da lista.
- Como o cabeçalho também reside no espaço heap, o espaço livre disponível é $4096 - 8 = 4088$.
 - Tamanho: 4088
 - Próximo: 0
 - 16384 espaço livre (4088 bytes)
 - Cabeçalho
 - Endereço virtual
 - Cabeça da lista livre

Incorporação da Lista Livre no Espaço Heap (Continuação)

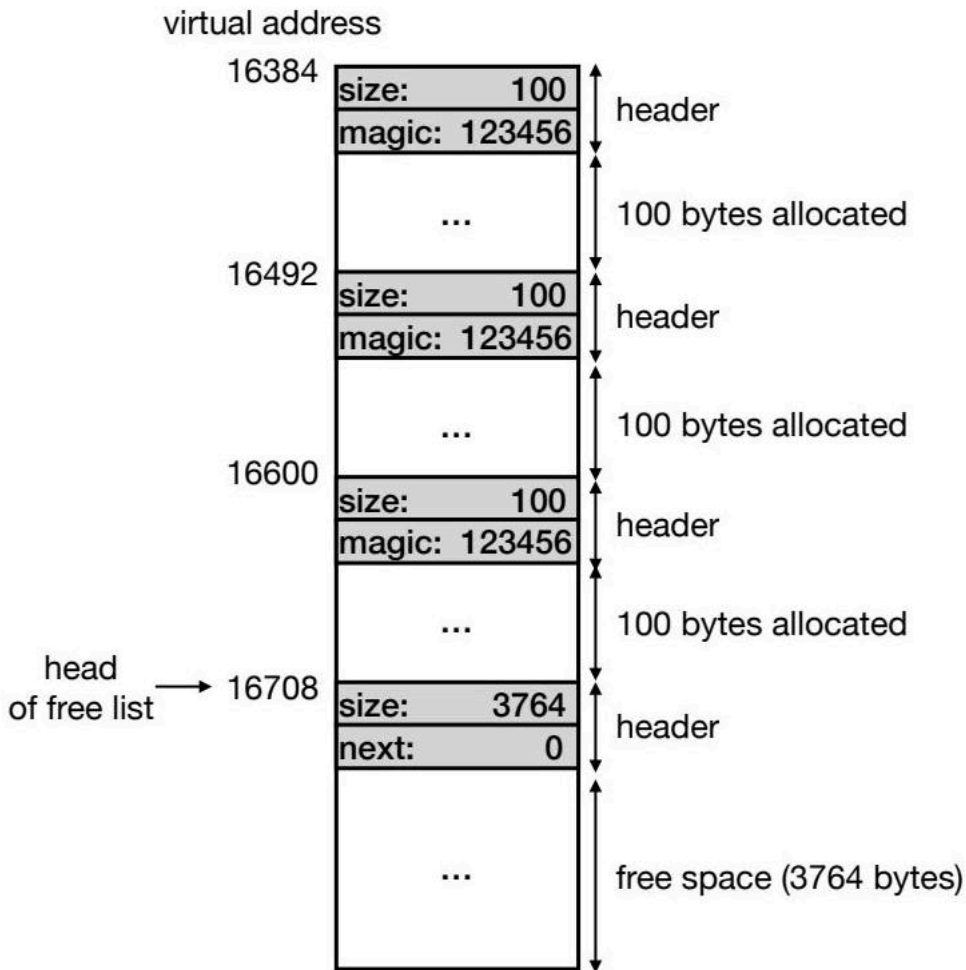
Uma solicitação `malloc()` para 100 bytes é emitida.

- O pedaço original é dividido em dois:
 - Um pedaço com 100 + 8 bytes (cabeçalho).
 - Como o espaço está sendo usado, o pedaço não é indexado pela lista livre (não há necessidade do campo `next`).
 - O campo `magic number` é usado para verificação de integridade.
 - Outro pedaço com 3980 + 8 bytes (cabeçalho).
- O ponteiro `head` (início da lista livre) é movido para o início do segundo cabeçalho.
 - Tamanho: 100
 - Magic: 123456
 - Espaço livre (3980 bytes)
 - Cabeçalho
 - Tamanho: 3980
 - Próximo: 0
 - ...
 - 100 bytes alocados
 - Cabeça da lista livre
 - Cabeçalho
 - 16384 endereço virtual
 - 16492

Incorporação da Lista Livre no Espaço Heap (Avançado)

Duas operações `malloc()` adicionais, cada uma solicitando 100 bytes, são emitidas.

- Nosso heap acaba com três pedaços alocados de 108 bytes e um único pedaço (3772 bytes) com espaço livre.
- E se `free()` for chamado para um determinado pedaço?

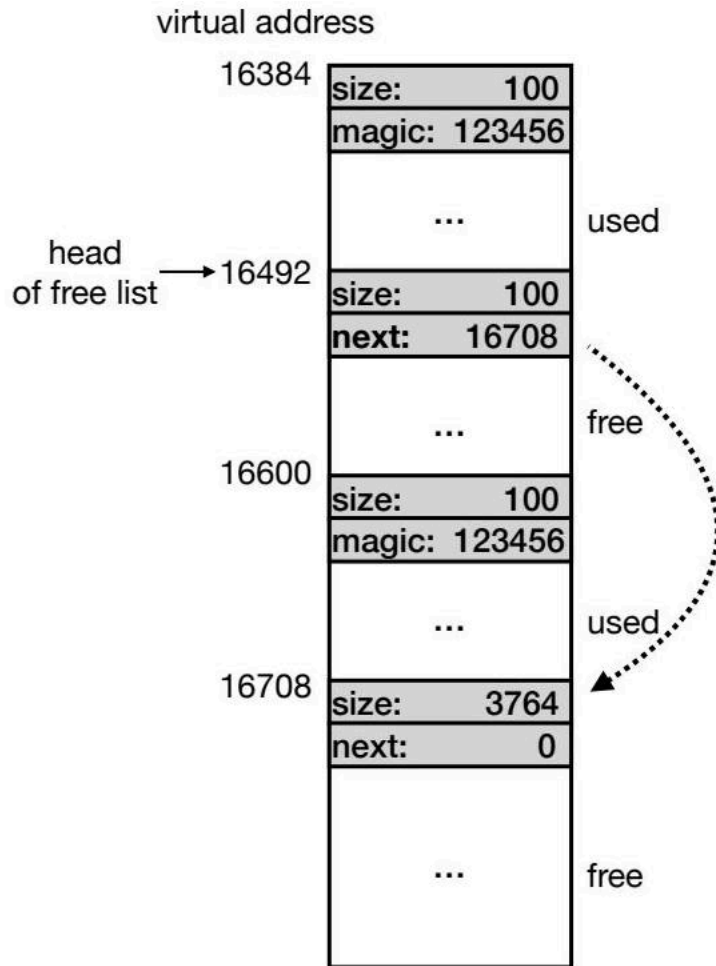


A imagem acima mostra a alocação de memória no heap, com blocos marcados como "usados" e um bloco "livre" com um cabeçalho indicando o tamanho disponível.

Incorporação da Lista Livre no Espaço Heap (Liberação de Memória)

`free()` é chamado para o endereço 16492 (segundo pedaço).

- Observação: `free()` não requer o tamanho da memória a ser liberada como argumento, porque a informação já está no cabeçalho.
- Supondo que se insere pedaços livres no início (`head`) da lista livre:
 - `Head` agora aponta para o pedaço recentemente liberado (endereço 16492).
- E se `free()` for chamado para os outros dois pedaços?



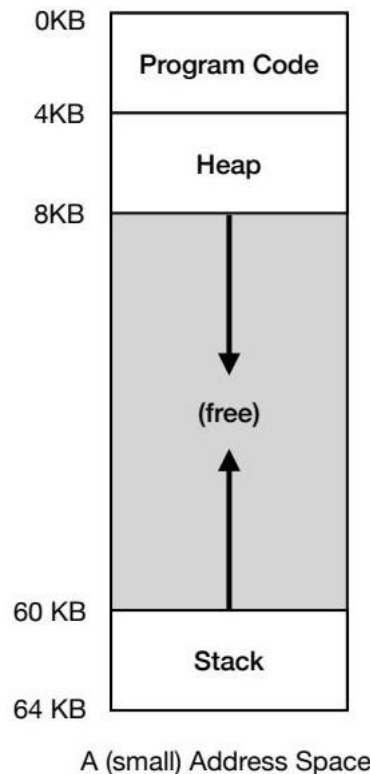
A imagem acima mostra a lista livre sendo atualizada após a liberação de um bloco, com o "head" apontando para o bloco recém-liberado.

Incorporação da Lista Livre no Espaço Heap (Coalescência Ideal)

`free()` é chamado primeiro para o endereço 16384 e depois para o endereço 16600.

- Todos os elementos estão agora livres.
 - **Head** agora aponta para o endereço 16600 (o último pedaço adicionado à lista livre).
- Problema: A lista livre agora tem vários fragmentos de pedaços.
 - Idealmente, seria preciso coalescer a lista livre, mesclando pedaços livres adjacentes em um único.
 - Quando terminado, o heap teria novamente um único e grande pedaço livre.
 - Tamanho: 100, Próximo: 16492
 - Tamanho: 100, Próximo: 16708
 - Tamanho: 100, Próximo: 16384
 - Tamanho: 3764, Próximo: 0
 - 16384 endereço virtual
 - 16492
 - 16600
 - 16708
 - Livre
 - Livre
 - Livre
 - Livre
 - Cabeça da lista livre

Solução de Problemas no Gerenciamento do Espaço Livre



A imagem acima ilustra a disposição do código do programa, heap e stack em um espaço de endereço.

O que fazer se não houver espaço no heap?

- Opção 1: O `malloc` falha.
- Opção 2: Aumentar o heap (começar com um pequeno e eventualmente aumentar seu tamanho).
 - É aqui que o alocador de memória faz uma chamada de sistema (por exemplo, `sbrk`).

Estratégias no Gerenciamento do Espaço Livre

Desafio: Qual elemento da lista livre o alocador de memória deve escolher para atender a um `malloc()` para 15 bytes?

- Vamos focar apenas no tamanho disponível (comprimento) em cada elemento da lista.
- Consegue pensar em estratégias?
 - Cabeça -> Comprimento: 10 -> Comprimento: 20 -> NULL
 - Lista Livre -> Comprimento: 30

Estratégia Best Fit

Percorrer a lista e encontrar o menor pedaço que se ajusta ao tamanho solicitado.

- A ideia é simples: encontrar o pedaço livre que melhor se ajusta à solicitação do usuário para evitar desperdiçar memória.
- Exemplo: A estratégia **best fit** percorre toda a lista e escolhe o comprimento 20.
 - Cabeça -> Comprimento: 10 -> Comprimento: 20 -> NULL
 - Lista Livre -> Comprimento: 30
- O resultado após a divisão seria:
 - Cabeça -> Comprimento: 10 -> Comprimento: 5 -> NULL
 - Lista Livre -> Comprimento: 30

Estratégia Worst Fit

Percorrer a lista e encontrar o maior pedaço que se ajusta ao tamanho solicitado.

- Ideia oposta ao **best fit**. O objetivo é usar pedaços grandes para que o espaço livre restante (após a divisão) ainda seja utilizável por outras solicitações de alocação.
- Estudos mostram que, de fato, a estratégia **worst fit** leva ao excesso de fragmentação externa.
- Exemplo: A estratégia **worst fit** percorre toda a lista e escolhe o comprimento 30.
 - Cabeça -> Comprimento: 10 -> Comprimento: 20 -> NULL
 - Lista Livre -> Comprimento: 30
- O resultado após a divisão seria:
 - Cabeça -> Comprimento: 10 -> Comprimento: 20 -> NULL
 - Lista Livre -> Comprimento: 15

Estratégia First Fit

As estratégias **best fit** e **worst fit** exigem percorrer toda a lista para encontrar um candidato. Isso pode levar algum tempo.

- É mais rápido encontrar um candidato, mas pode poluir o início da lista livre com vários pequenos pedaços livres (levando à grave fragmentação externa dos primeiros pedaços na lista livre).
- Exemplo: A estratégia **first fit** escolheria o comprimento 30, sem percorrer toda a lista.
 - Cabeça -> Comprimento: 10 -> Comprimento: 20 -> NULL
 - Lista Livre -> Comprimento: 30
- O resultado após a divisão seria:
 - Cabeça -> Comprimento: 10 -> Comprimento: 20 -> NULL
 - Lista Livre -> Comprimento: 15

Estratégia Next Fit

A estratégia **next fit** procura o primeiro pedaço livre que seja grande o suficiente para acomodar a solicitação.

- Diferentemente do **first fit**, ela registra o último pedaço sendo dividido na lista livre. Quando uma nova alocação é necessária, a busca começa a partir dessa posição em diante.
- A busca por pedaços livres torna-se mais uniforme em toda a lista livre, evitando pequenos fragmentos no início da lista.
- Exemplo: A estratégia **next fit** escolheria o comprimento 30, sem percorrer toda a lista. Se então outra solicitação para alocar 5 bytes chegasse, a busca começaria a partir do segundo pedaço em diante.

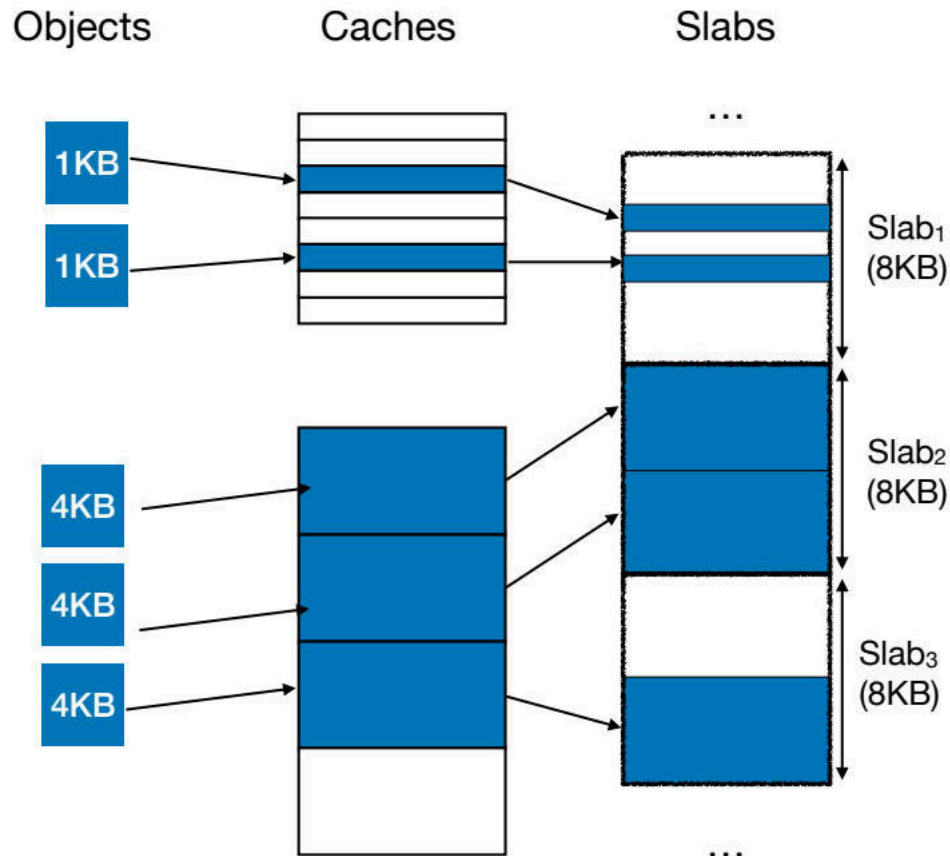


A imagem acima demonstra o estado da lista livre após múltiplas divisões, com diferentes blocos de memória alocados.

Alocador Slab e Listas Segregadas

Se um programa tem uma (ou algumas) solicitações de alocação de memória de tamanho popular, mantenha listas livres separadas (segregadas) para gerenciar objetos com esses tamanhos.

- A fragmentação é fácil de gerenciar.
- Encontrar espaço livre é muito eficiente.
- Interessante para objetos do kernel (por exemplo, o alocador slab).



A imagem acima ilustra a alocação de objetos em caches, que por sua vez são gerenciados por slabs, mostrando como diferentes objetos de tamanho são acomodados em slabs de tamanho fixo.

- Diferentes caches de objetos são alocados para objetos específicos do kernel (por exemplo, locks, inodes do sistema de arquivos, ...).
- Cada cache de objeto é uma lista livre segregada com um determinado tamanho de objeto.

Quanta memória deve-se dar a cada cache?

- Slabs, gerenciados por um alocador de memória mais geral, permitem tornar isso uma decisão dinâmica, alocando um slab para um cache quando este está com pouca memória disponível e realocando um slab quando o cache não está mais usando.

Disclaimer: exemplo ilustrativo.

Alocação Buddy

Na alocação buddy, a memória livre é gerenciada como um grande espaço de tamanho 2^N . Quando uma solicitação de memória é feita, o espaço livre é recursivamente dividido por dois até atingir o menor pedaço que pode conter a solicitação.

- Os pedaços têm um tamanho fixo (2^N), o que significa que este esquema pode sofrer de fragmentação interna!
- Exemplo: Alocar 5KB em 64KB

Divisão na Alocação Buddy

- Memória livre é gerenciada como um grande espaço de tamanho 2^N .
- Quando uma solicitação de memória é feita, o espaço livre é recursivamente dividido por dois até atingir o menor pedaço que pode conter a solicitação.
- Pedaços têm um tamanho fixo (2^N), o que significa que este esquema pode sofrer de fragmentação interna!
- Exemplo: Alocar 5KB

64KB

32KB

16KB

Coalescência na Alocação Buddy

☉ The algorithm shines when coalescing blocks

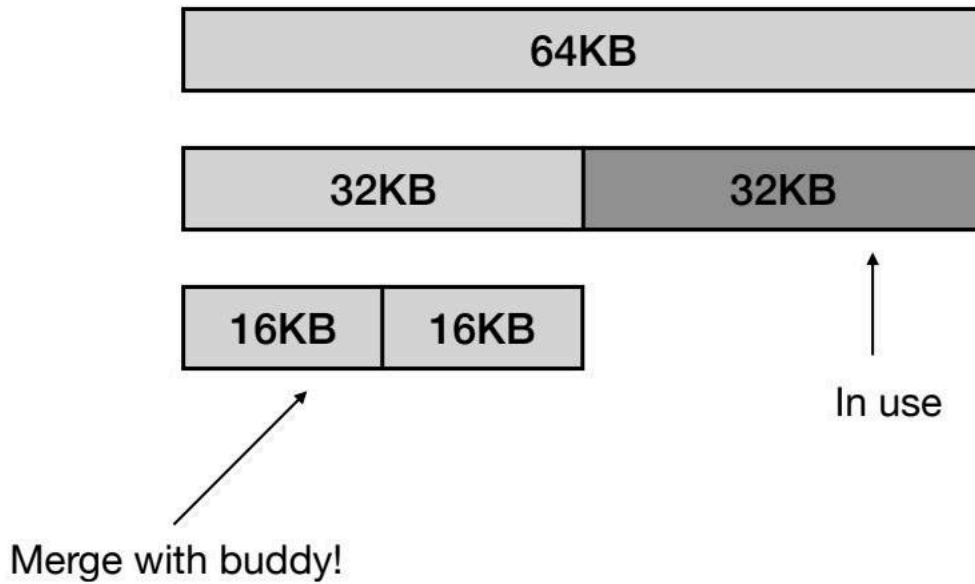
- ▶ When a block is free, and the buddy (neighbor) is a free, then **coalesce the two chunks**
- ▶ The resulting chunk is then matched with its buddy coalesced again if both are free (**recursive pattern**)

☉ **Example:** Free the first 8KB block

A imagem acima demonstra como blocos de memória adjacentes são combinados em um único bloco maior, aumentando a disponibilidade de memória contígua, seguindo o algoritmo buddy.

O algoritmo se destaca ao coalescer blocos.

- Quando um bloco está livre e o buddy (vizinho) também está livre, então coalescer os dois pedaços.
- O pedaço resultante é então combinado com seu buddy e coalescido novamente se ambos estiverem livres (padrão recursivo).
- Exemplo: Liberar o primeiro bloco de 8KB



A imagem acima ilustra como blocos de memória são marcados para serem mesclados com seus buddies adjacentes.

Outros Alocadores

Usar listas pode não ser a estrutura mais eficiente.

- Alocadores avançados usam estruturas mais complexas, como árvores.
- Liberar memória para cargas de trabalho multithread e sistemas multiprocessadores também é desafiador para os alocadores.
- Fazer um alocador rápido, com uso eficiente de espaço e escalável ainda é um desafio de pesquisa em nossos dias :)

Mais Informações

- Capítulos 13 a 17 - Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. Operating Systems: Three Easy Pieces. Arpaci-Dusseau Books, 2018.
- Avi Silberschatz, Peter Baer Galvin, Greg Gagne. Operating System Concepts (10. ed). John Wiley & Sons, 2018.

Alguma pergunta?